Connected Components - using DFS

We can find the number of connected components in a graph using the Depth-First Search (DFS) traversal algorithm.

Here's a simple pseudocode for finding the number of connected components using DFS in an undirected graph (same algorithm can be used for directed graphs as well):

```
procedure DFS(node, visited)
    mark node as visited
    for each neighbor in neighbors of node
        if neighbor is not visited
            DFS(neighbor, visited)

procedure FindConnectedComponents(graph)
    initialize connected components counter with 0

    for each node in graph
        if node is not visited
            DFS(node, visited)
            increment connected components counter by 1

    return connected components counter
```

Explanation:

- The `DFS` procedure recursively explores the graph starting from a given node. It marks each visited node . It then continues the exploration to its neighbors.
- The `FindConnectedComponents` procedure initializes a counter variable to 0 which stores the count of connected components. It iterates through each node in the graph. If the node is not visited, it calls the `DFS` procedure to explore all nodes in that component and increment the connected component counter by 1.

Task

- Implement the above given algorithm to find out the no. of connected components in given undirected graph.
- Just output the count of connected component, not the actual nodes in each components.

Constraints

- $1 \leq N \leq 200000$
- $1 \leq M \leq N(N-1)/2$
- $1 \leq u_i, v_i \leq N$
- $u_i \neq v_i$ for each $1 \leq i \leq M$.

Sample 1:

Input	Output
5 6 1 2 2 3 1 3 3 5 2 4 4 5	1

More Info

Time limit

1 secs

Memory limit

1.5 GB

Source Limit

50000 Bytes